Removing the VDM: 3D-Tracking Geometric State Judgment for Code-as-Policies Robot Agents

cap-x-se / track-judge experiment

June 23, 2026 (living draft — figures/tables auto-generated by `make_figs.py`)

Abstract

Agentic Code-as-Policies systems call a Visual Differencing Model (VDM)—a VLM—every turn to judge task state and steer the agent's self-correction. We argue the VDM is a *redundant LLM call*: a second VLM invocation whose errors are correlated with the policy LLM, returning largely the agent's own opinion at real latency and cost. We *remove* it and introduce a *self-evaluating* agent that self-plans *and* self-evaluates—both as code, with *no LLM call* at judgment time and free use of any packages—basing its state judgment on agent-written geometric code over *3D point trajectories* (TAPIP3D), a decorrelated, physically grounded, local signal. On libero-pro with a single Gemini cap-agent0, we test whether this makes the agent both more successful and faster. [**Result: TBD — this is a living draft.**]

1 Introduction

The per-turn VDM in cap-x-se is an extra VLM call. Because it shares the policy LLM's modality and family, its judgments are correlated with the policy's own beliefs, so it adds little *independent* information while costing a full inference each turn. We ask whether the VDM can be removed entirely and replaced with a fundamentally different signal: deterministic geometry over the 3D trajectories of tracked scene points. The agent, given the same initial static frame, writes *two* artifacts—execution code and a state-judgment function; the latter runs on the dense RGB-D rollout video through a 3D point tracker and decides progress/done/failure geometrically. Our thesis is that this is **both faster** (no per-turn VLM; early abort on geometric failure) **and more successful** (a richer, grounded, decorrelated signal).

2 Why a 3D trajectory beats a frame-diff VLM

A VDM returns a discrete 2D semantic snapshot; a tracker returns continuous metric 3D trajectories of chosen masks. The latter affords judgments the VDM structurally cannot make:

- **Collision / near-miss**: minimum inter-object distance over time (cm).
- **Grasp slip**: object points tracked *relative to the gripper*; loss of co-motion ⇒ slip, caught mid-motion.
- **Early failure → early abort**: detect failure during execution, retry immediately (primary speed and success lever).

- **Drop detection**: descent past support height + free-fall signature.
- **3D containment / placement**: points entering the target container's 3D volume, persistent across the last K frames.
- **Occlusion robustness**: occluded points' positions are predicted by the tracker.
- **Appearance/viewpoint invariance**; **multi-object relations** (on-top-of, inside, aligned); **determinism/debuggability**; **dense, cheap, graded progress**; "right motion, wrong outcome" (missed grasp).

3 Method

Agent. cap-agent0: a single-LLM (Gemini) Code-as-Policies loop that writes Python executed against bound robot APIs, with multi-turn REGENERATE/FINISH self-correction. **Arm A (baseline):** the stock VDM. **Arm B (treatment):** VDM removed; per-turn feedback comes from the agent-written geometric judge over TAPIP3D 3D tracks (§2), which selects its own masks. The two arms differ only in the feedback mechanism (a single config switch); the agent, LLM, suites, seeds and init states are identical.

Tracking judge. The rollout is recorded as a dense RGB-D video with known camera intrinsics/extrinsics (exact in simulation); per turn it is tracked to 3D point trajectories in the world frame, and the agent's judge function applies geometric primitives to emit {progress, done, failure, abort}. All tracking is visualized and saved for debugging.

4 Experimental setup

Benchmark: libero-pro. We use six suites (object/goal/spatial, each in a *swap* and a *task* perturbation), **10 tasks per suite with 50 initial states each — 3000 trials per arm in total**. Evaluating all 3000 per arm every self-evolve iteration is intractable (each trial is a multi-turn, multi-LLM-call rollout), so experiments run on a **fixed sampled subset** (a capped number of tasks/suite and initial-states/task, recorded in each run's `summary.json` so a subsampled run is never reported as the full set). Success is ground-truth `check_success()`. A dev subset is used to iterate; ≥ 2 suites are held out for generalisation and never tuned on. **Metrics:** task success rate and median wall-clock per task, reported together. **Protocol:** an autonomous self-evolve loop optimises Arm B; the win condition is to beat Arm A on *both* metrics. Arm B is scored as the pure method (no VDM/ground-truth backstop).

5 Results

Living section — regenerated on every accepted generation.

Table 1: Headline: VDM baseline (A) vs. tracking with the VDM removed (B), dev suites.

metric	VDM (A)	tracking (B)
success rate	–	–
median task time (s)	–	–
judge latency / turn (s)	–	–
VLM calls / turn	–	–

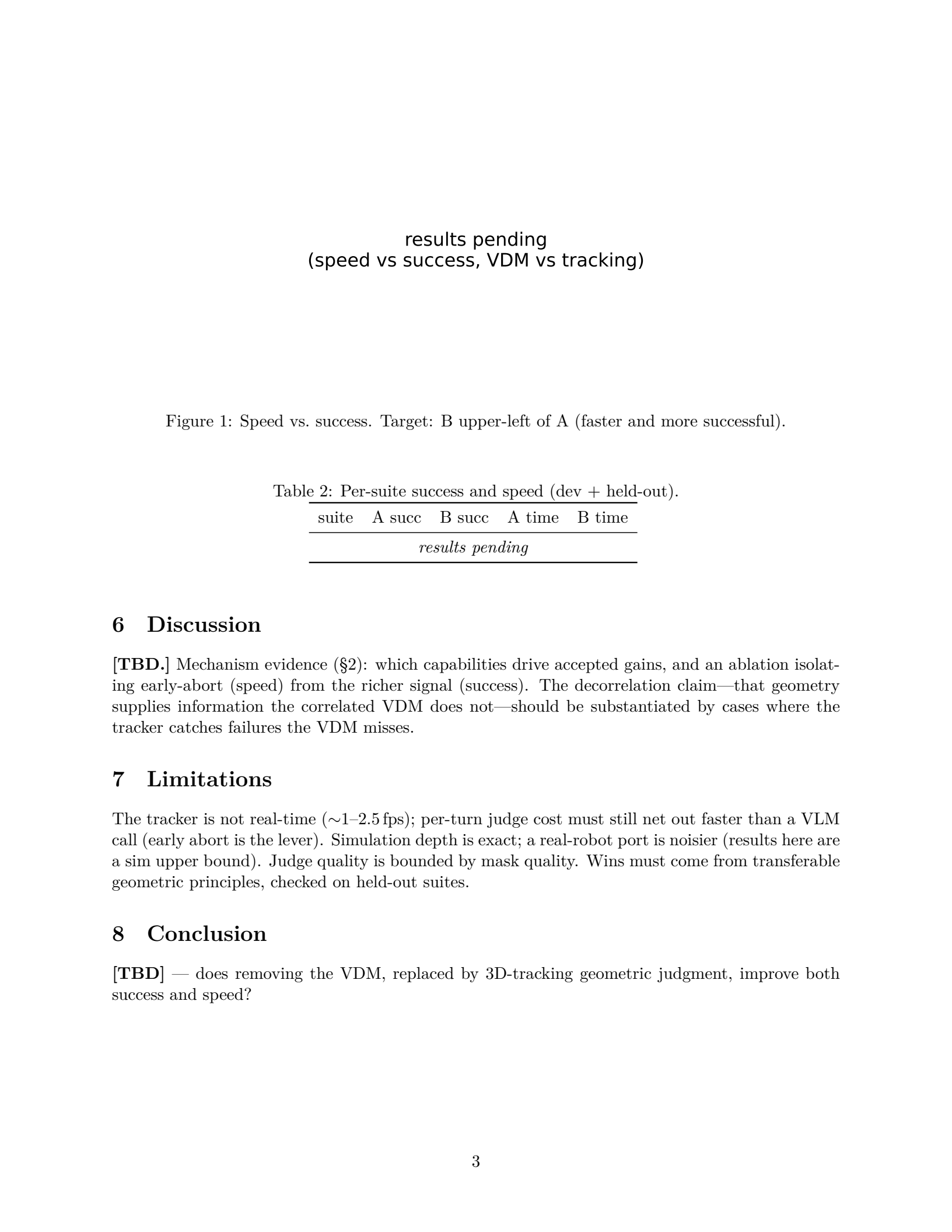

Figure 1: Speed vs. success. Target: B upper-left of A (faster and more successful).

Table 2: Per-suite success and speed (dev + held-out).

suite	A succ	B succ	A time	B time
results pending				

6 Discussion

[**TBD.**] Mechanism evidence (§2): which capabilities drive accepted gains, and an ablation isolating early-abort (speed) from the richer signal (success). The decorrelation claim—that geometry supplies information the correlated VDM does not—should be substantiated by cases where the tracker catches failures the VDM misses.

7 Limitations

The tracker is not real-time ($\sim$1–2.5 fps); per-turn judge cost must still net out faster than a VLM call (early abort is the lever). Simulation depth is exact; a real-robot port is noisier (results here are a sim upper bound). Judge quality is bounded by mask quality. Wins must come from transferable geometric principles, checked on held-out suites.

8 Conclusion

[**TBD**] — does removing the VDM, replaced by 3D-tracking geometric judgment, improve both success and speed?

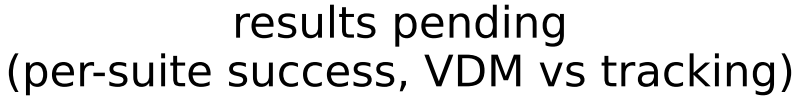

Figure 2: Per-suite success rate, VDM (A) vs. tracking (B).